ELO320 - Estructuras de Datos y Algoritmos
Tipos de Datos Abstractos
Árboles Binarios y Recursividad

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

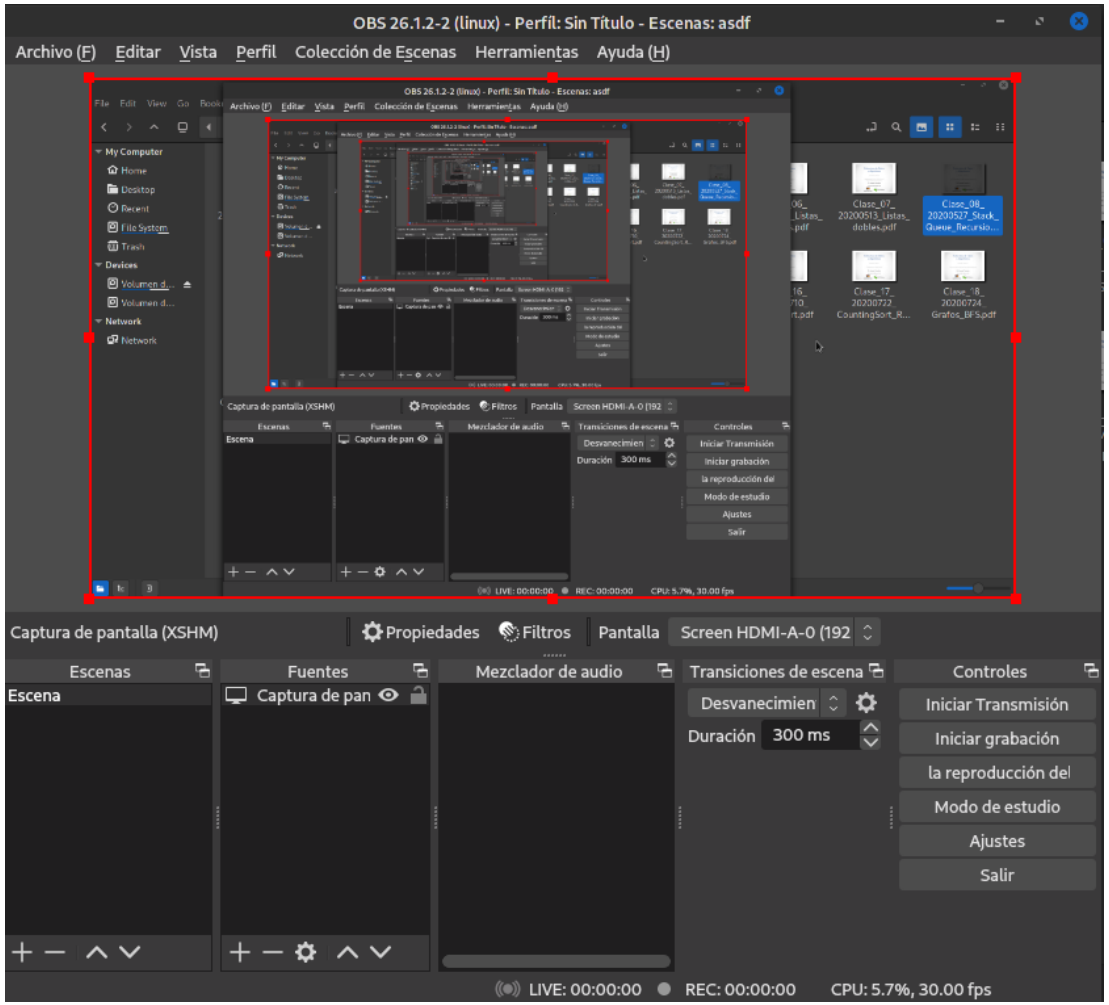
Ing. Civil Telemática - Departamento de Electrónica

1er Semestre 2025

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Recursividad

Los procesos recursivos son aquellos ciclos autorreferentes, vale decir, una definición (matemática, funciones, programas) depende parcialmente de sí misma.



Recursividad

Una definición recursiva se compone de dos elementos básicos:

1. Un caso base que detiene el ciclo.
2. Una llamada autorreferente con un elemento.

Recursividad

Factorial

$f(0) = 1$
$f(n) = n \times f(n-1)$.

Factorial en C

```
int factorial(int n){
        if (n<0)
                return -1; //Valor de Error
        elif(n==0) // caso base, sin condicion -> loop infinito
                return 1;
        return n * factorial(n-1); // llamado a un elemento parcial del caso
}
```

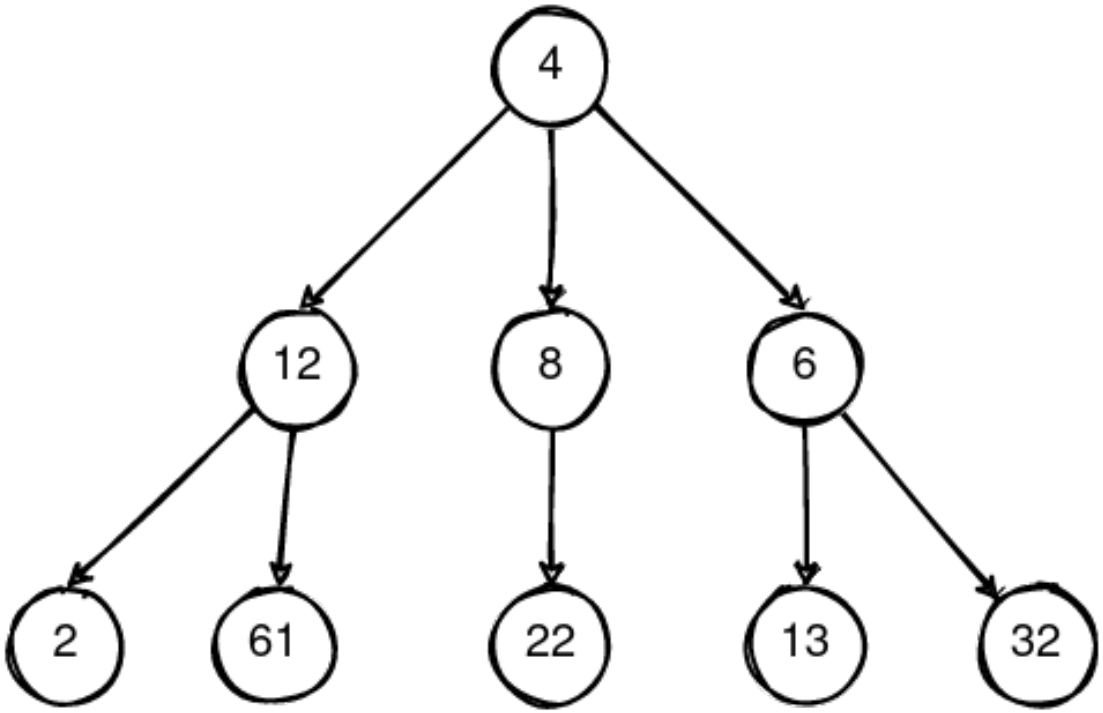
Entenderemos por recursivdad como la técnica en la cual alguna función depende de su propia definición, vale decir, **se llama a sí misma** para formar un ciclo.

Recursividad: Ejemplos

1. Sequencia de Fibonacci.
2. Polinomio de Hermite.
3. Triángulo de Pascal.
4. Fractales.
   - Curva de Peano.
   - Curva de Koch.
   - Curva de Hilbert.
5. **Árboles**.

Árboles

Los árboles son una estructura de dato no-lineal, jerárquica y enlazada que permite acceso eficiente a los elementos de la estructura tanto como a actualización de la misma.



Árboles: Aplicaciones

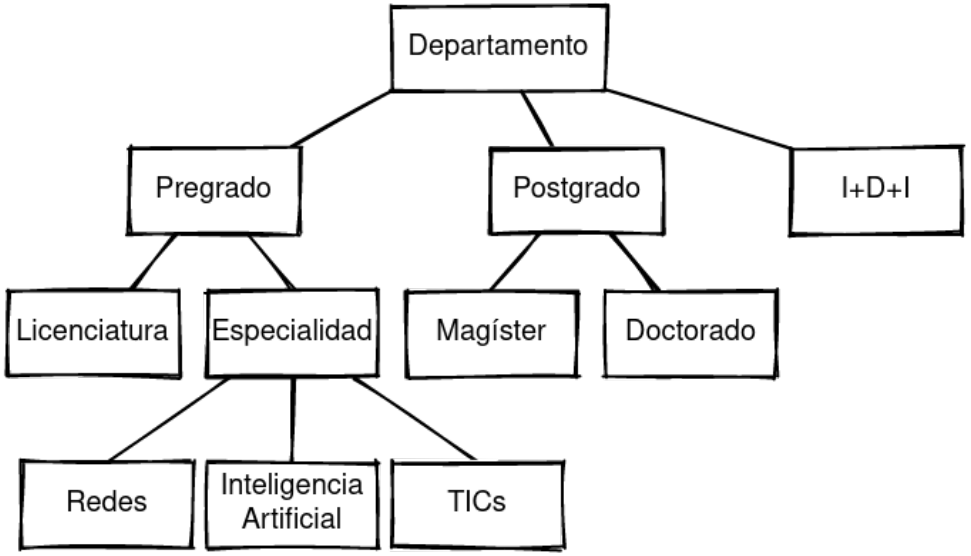
Los árboles son extensamente utilizados en diversas áreas de las telecomunicaciónes, tecnologías de la información, matemáticas, electrónica e informática:

- Compilación e interpretación de elementos sintácticos de un lenguaje.
- Representación expresiones aritméticas y/o lógicas.
- Representación de estructuras de búsqueda de datos.
- Sistemas de archivos y directorios.
- Clases y herencia en POO.
- Compresión de datos.
- Computación Evolutiva.

Árboles: Conceptos

Un árbol está formado por un conjunto finito de elementos, llamados nodos los cuales son enlazados entre sí según jerarquía padre-hijo.

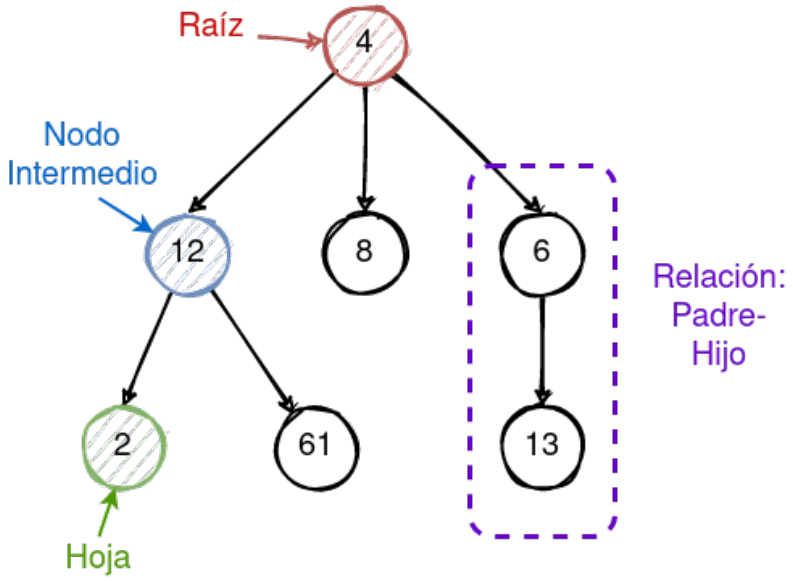
- Primer nodo → raíz del árbol.
- Relación padre-hijo → jerarquía.
- Nodos → varios sucesores.
- Nodos → único antecesor.



Árboles: Conceptos

Los componentes de los árboles son nombrados y clasificados según sus características.

- Primer nodo → raíz del árbol.
- Nodo con sucesores → padre.
- Sucesor de un nodo → hijo.
- Nodos con padre e hijos → nodo intermedio.
- Nodo sin hijos → hoja (nodo terminal).



Árboles: Conceptos

Además en un árbol, se identifica la siguiente información:

- Camino: Secuencia entre dos nodos siguiendo relación jerárquica padre hijo.

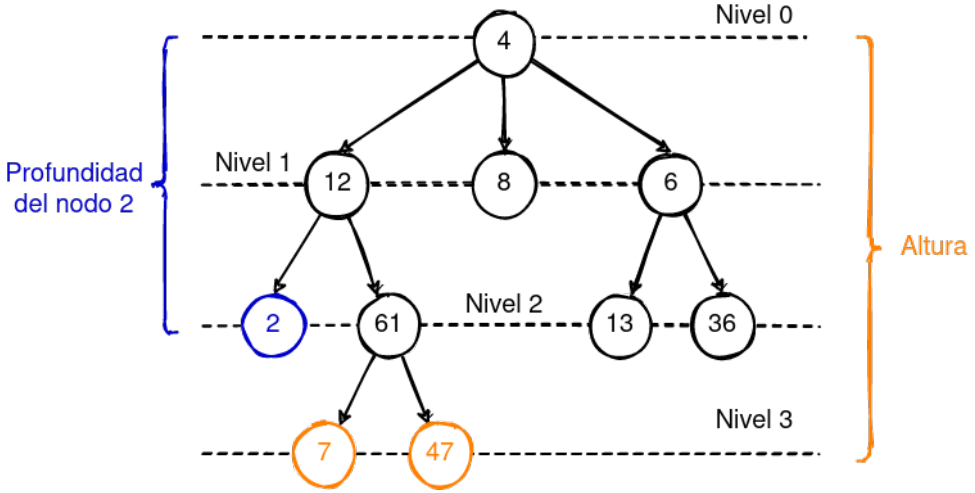
$$\text{camino}(4, 47) = \{4, 12, 61, 47\}$$

- Largo Camino: Número de enlaces (nodos-1) entre dos nodos en relación jerárquica.

$$\text{largo}(4, 47) = 3$$

- Rama: Camino de la raíz a una hoja.

$$\text{rama}_1 = \text{camino}(4, 2)$$



Árboles: Conceptos

- Grado de Nodo: Cantidad de hijos.

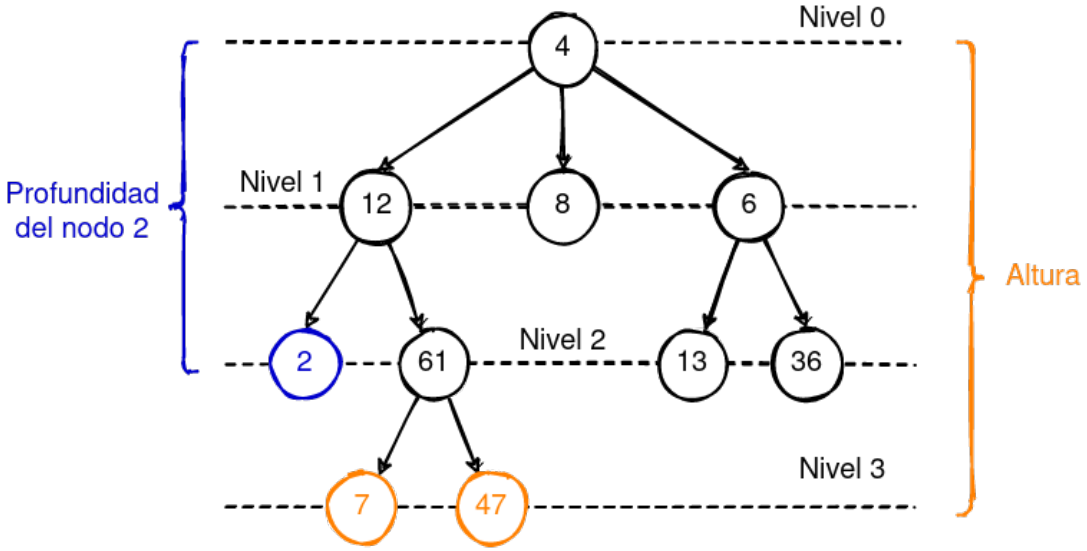
$$\text{grado}(47) = 0$$

- Grado del árbol: Máximo número de hijos para cada nodo.

$$\text{grado}(A) = 3$$

- Profundidad: Cantidad de predecesores del nodo.

$$\text{profundidad}(2) = 2$$



Árboles: Conceptos

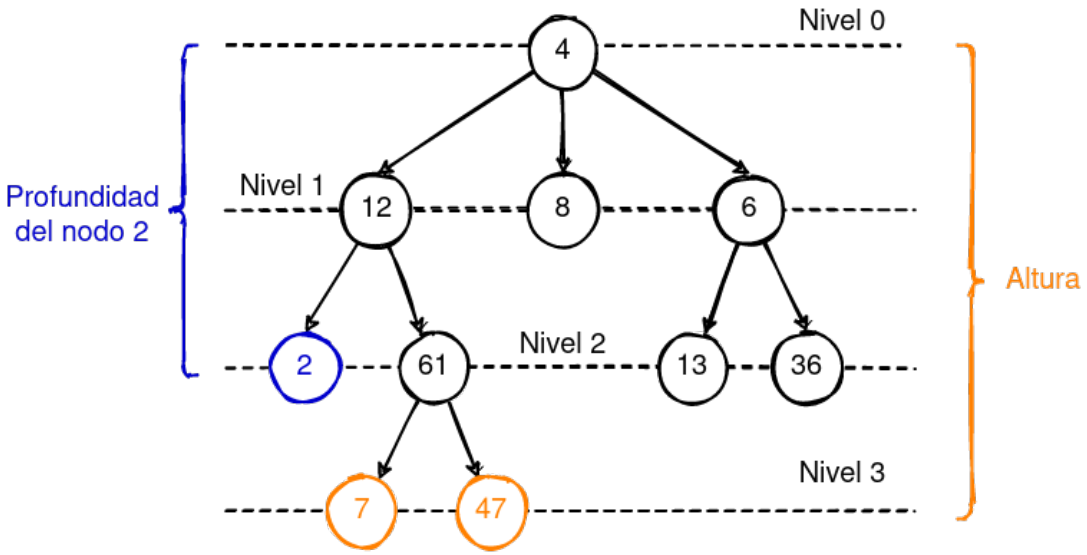
- Altura: Profundidad Máxima de los nodos o largo del camino más grande.

$$\text{altura}(A) = 3$$

- Nivel: Número de enlaces del camino de la raíz al nodo (parte en 0).

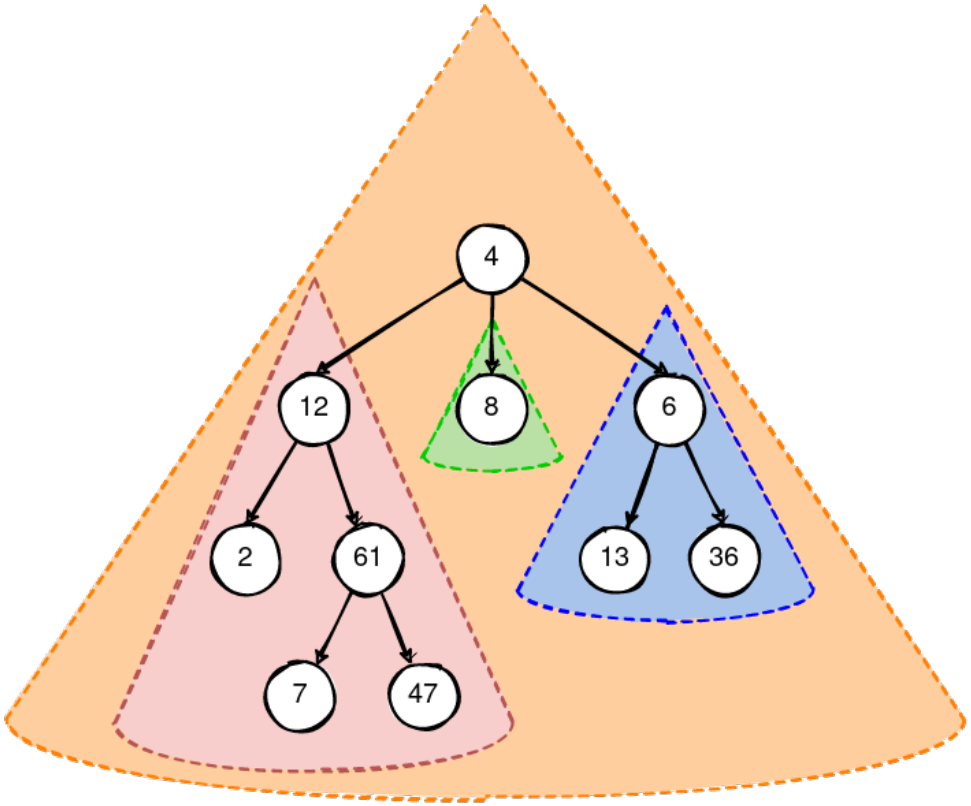
$$\text{nivel}(4) = 3$$

- Bosque: Colección o conjunto de árboles.



Árboles: Recursividad

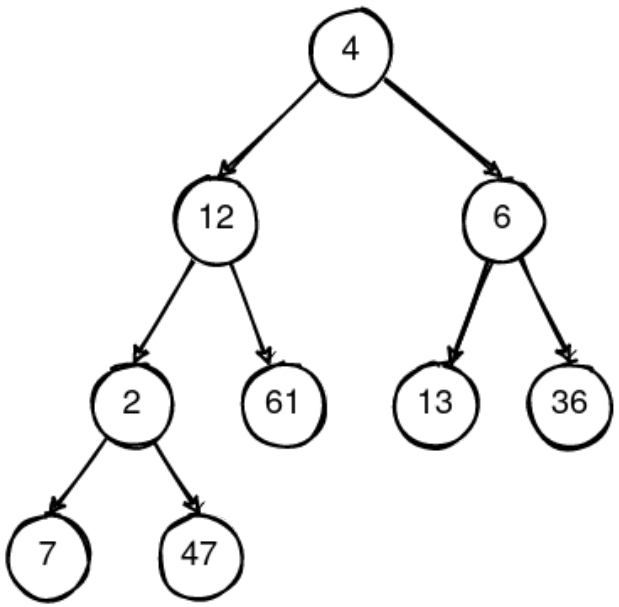
Los árboles son estructuras de datos recursiva, debido a que cada hijo de un nodo es la raíz de un árbol de menor altura (sub-árbol).



Por esta razón, las funciones y métodos de árboles son facilmente aplicables con técnicas recursivas.

Árboles binarios

En este curso nos concentraremos en árboles binarios, i.e., árboles cuyo grado es 2.

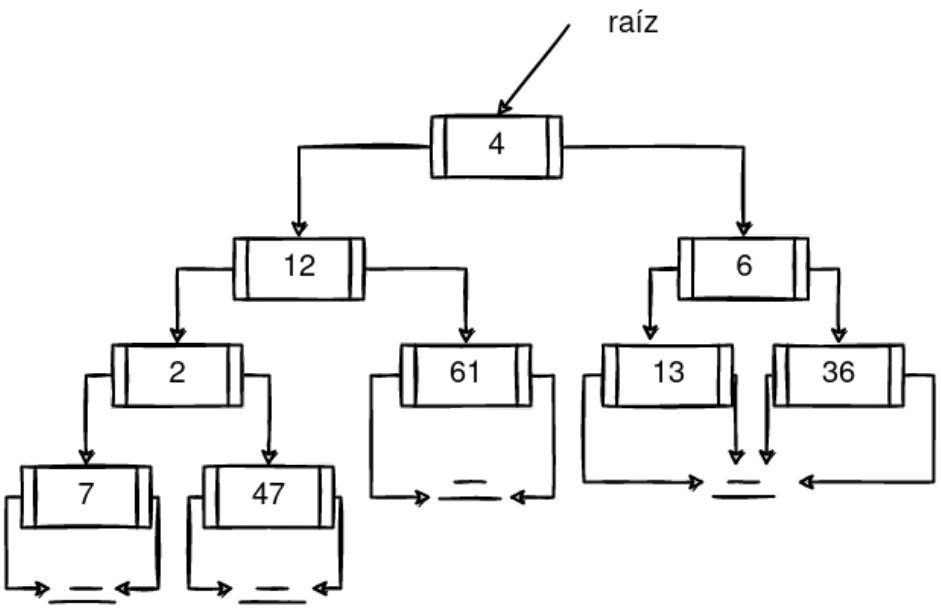


Aplicaciones:

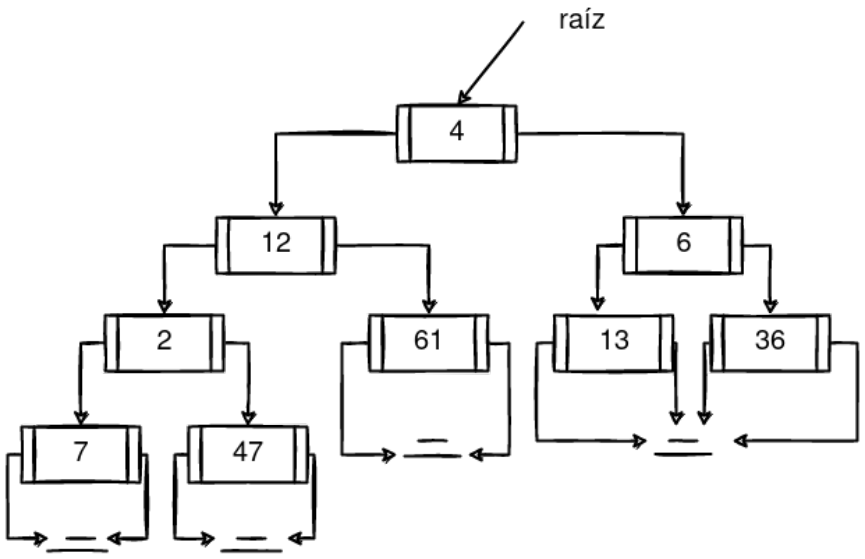
- Gramática de lenguajes, i.e. matemáticas.
- Árboles de Decisión.
- **Árbol Binario de Búsqueda.**

Árboles binarios: Implementación

Un árbol binario tiene a lo más dos hijos, y se implementa muy similar a las listas enlazadas.



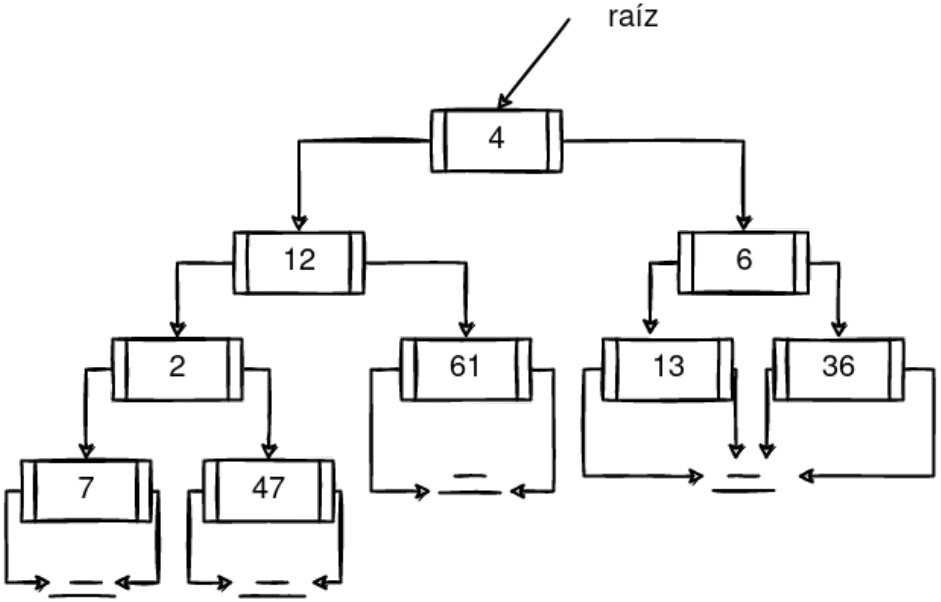
Árboles binarios: Implementación



Árbol Binario en C

```
typedef struct nodo{
        int dato;
        struct nodo *hijo_izq;
        struct nodo *hijo_der;
} nodo_t;

int main(int argc, char **argv){
        nodo_t *raiz; // árbol vacío
        return 0;
}
```

Árboles binarios: Implementación



Detalles:

- El puntero de control se encuentra siempre en la raíz.
- Los nodos cuyos hijos apunten a `NULL`, son hojas.
- **¿Cómo podemos recorrer un árbol (binario)?.** - En seguida.
- **¿Cómo agregar, buscar, eliminar elementos de un árbol binario?** - ABB: Próxima clase.

Árboles binarios: Recorridos

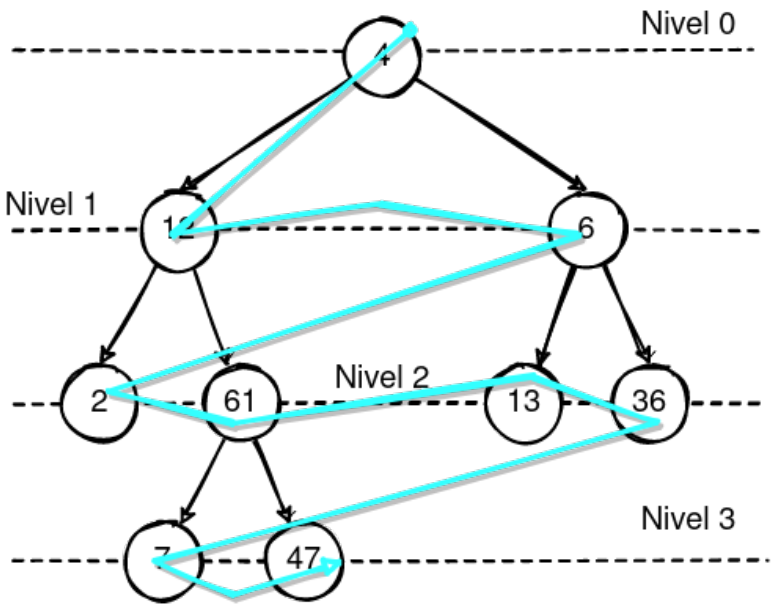
Muchas aplicaciones necesitan procesar un árbol visitando sus nodos, y realizando acciones sobre ellos: *mostrar su contenido*, *sumar valores guardados*, *encontrar un nodo específico*, etc.

Explicaremos dos de las formas más clásicas para recorrer un árbol:

- Recorrido en anchura.
- Recorrido en profundidad.

Árboles binarios: Recorrido en Anchura

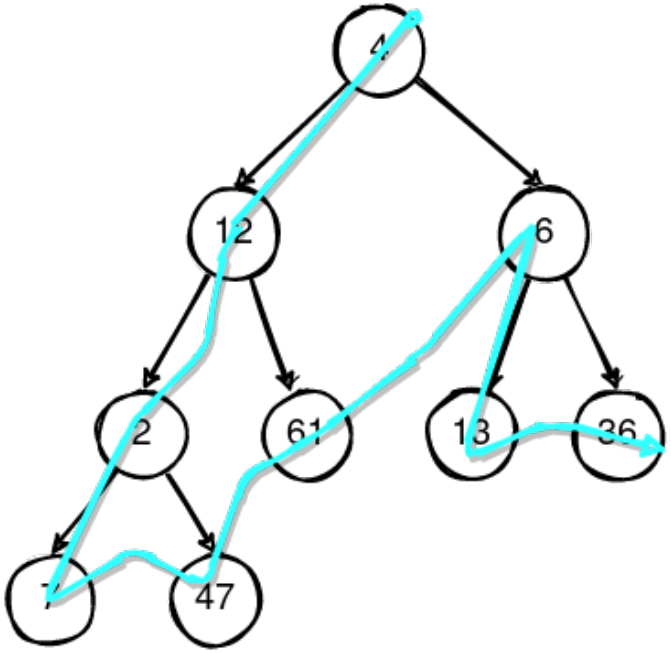
Tiene como criterio visitar todos los nodos de un nivel antes de pasar al siguiente.



- No puede realizarse directamente por recursividad.
- Se utiliza una **Queue/Cola** como estructura auxiliar.
- Recorre todos los nodos de un nivel, antes de pasar al siguiente.
- Se realiza dequeue y enqueues hasta pasar por todos lo niveles.

Árboles binarios: Recorrido en Profundidad

Tiene como criterio visitar los nodos por profundidad hasta llegar una hoja, luego retrocede y continúa.



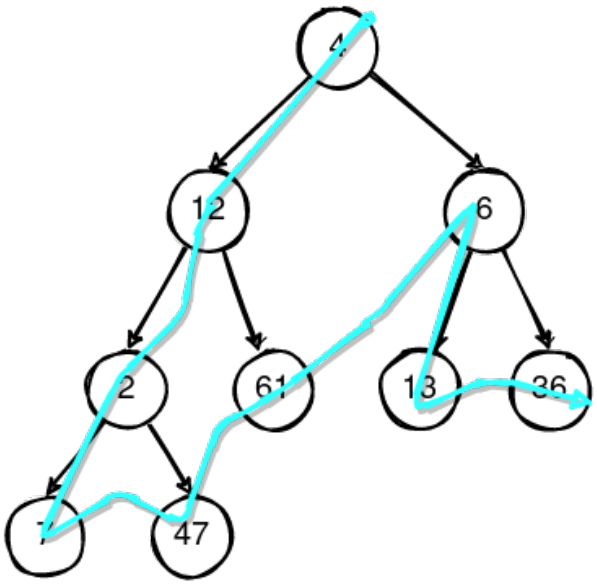
Tres de los tipos de recorrido en profundidad más utilizados son:

- Pre-orden.
- In-orden.
- Post-orden.

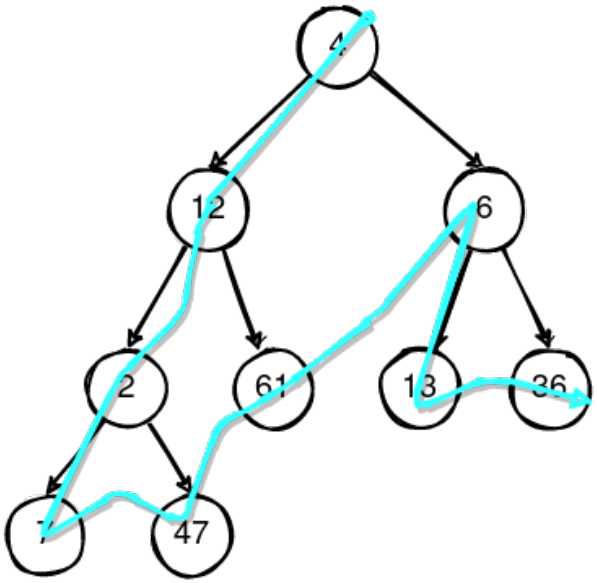
Árboles binarios: Rec. Prof. Pre-orden

Recorrido recursivo que sigue la siguiente orden de visitas:

1. Visitar nodo actual.
2. Visitar hijo izquierdo en pre-orden.
3. Visitar hijo derecho en pre-orden.



Árboles binarios: Rec. Prof. Pre-orden



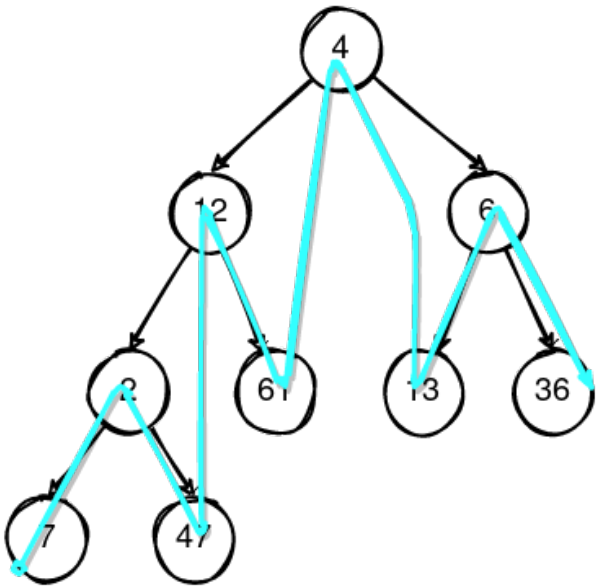
Pre-orden en C

```
void print_preorden(node_t *root){
        printf("Dato: %d\n", root->dato);

        if(root->hijo_izq != NULL)
                print_preorden(root->hijo_izq);

        if(root->hijo_der != NULL)
                print_preorden(root->hijo_der);
}
```

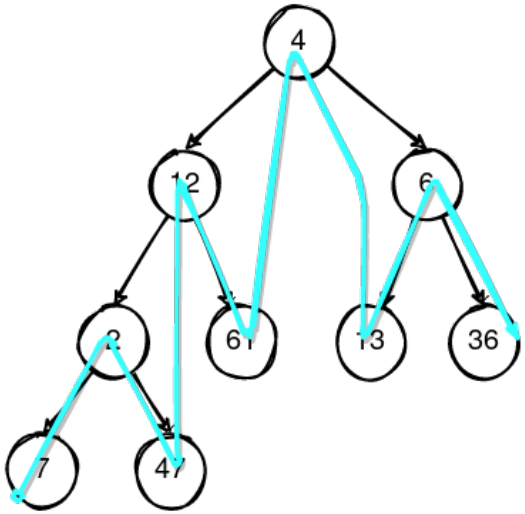
Árboles binarios: Rec. Prof. In-orden

Recorrido recursivo que sigue la siguiente orden de visitas:

1. Visitar hijo izquierdo en in-orden.
2. Visitar nodo actual.
3. Visitar hijo derecho en in-orden.



Árboles binarios: Rec. Prof. In-orden



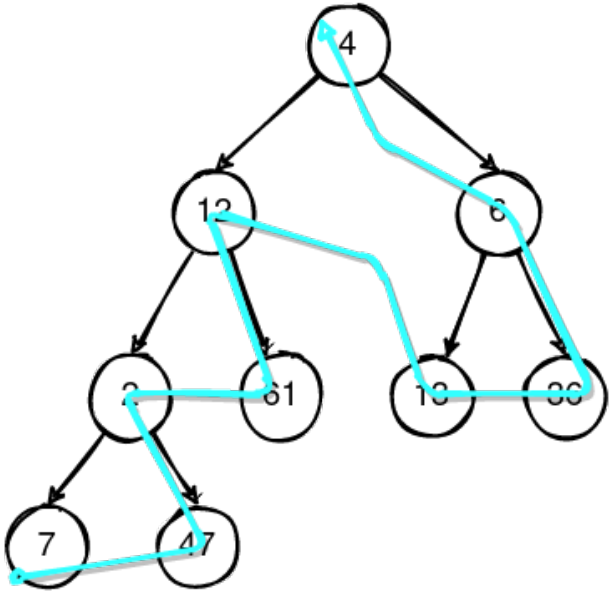
In-orden en C

```
void print_inorden(node_t *root){

        if(root->hijo_izq != NULL)
                print_inorden(root->hijo_izq);

        printf("Dato: %d\n", root->dato);


        if(root->hijo_der != NULL)
                print_inorden(root->hijo_der);
}
```

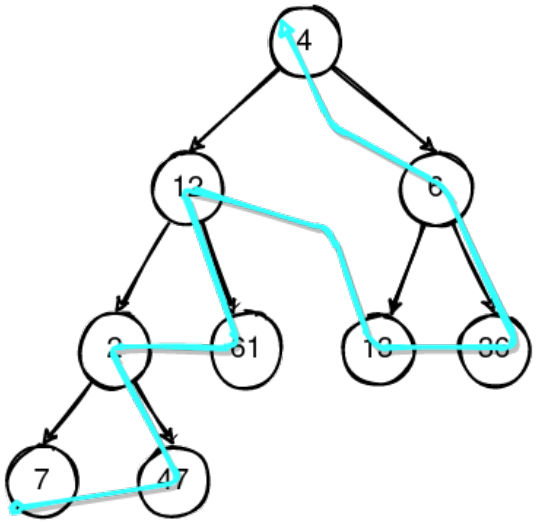
Árboles binarios: Rec. Prof. Post-orden

Recorrido recursivo que sigue la siguiente orden de visitas:

1. Visitar hijo izquierdo en post-orden.
2. Visitar hijo derecho en post-orden.
3. Visitar nodo actual.



Árboles binarios: Rec. Prof. Post-orden



Post-orden en C

```
void print_postorden(node_t *root){

        if(root->hijo_izq != NULL)
                print_postorden(root->hijo_izq);

        if(root->hijo_der != NULL)
                print_postorden(root->hijo_der);

        printf("Dato: %d\n", root->dato);
}
```